

Condition Editor

An editor for creating and editing conditions. The conditions are broken down and displayed as a structure. In its simplest form, an expression will consist of only one condition; however, more complex expressions with one or more logical groups are also quite common. The structural representation is meant to provide a clean and comprehensive overview, even for more complex conditions. This also makes it easy to make changes, whether they are small value changes or larger logical changes. The structure makes it easy to keep track of parentheses and the order of execution.

✕

Expression

AND

AND

DatasetId != null

InputType != null

OR

TaskOption = "Now"

OR

AND

TaskOption = "Schedule"

ScheduleName != null

AND

ScheduleOption = "Daily"

ScheduleTime != null

AND

ScheduleOption = "Weekly"

ScheduleTime != null

ScheduledDays != null

AND

ScheduleOption = "Monthly"

ScheduleTime != null

ScheduledDayNumber > 0

Apply

Cancel

Conditions

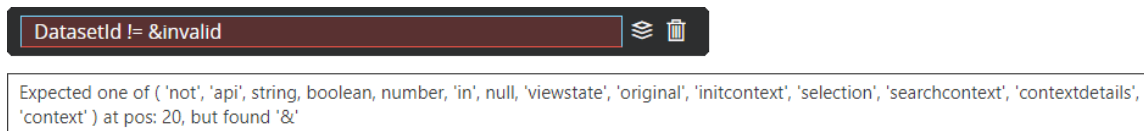
In the boolean expression structure, the separate sub-conditions are written as free text in the format of \ [operator \]. Operands can be literals, attributes, or functions. A sub-condition can not contain logical operators. For condition syntax details, see [Condition Syntax](#)



	Description
1	Drag handle, can be used to re-locate a sub-condition
2	Duplicates a sub condition
3	Deletes a sub condition

Condition Validation

When editing a sub-condition, the input is validated as you type. If the validator finds errors, the field will display as having errors. Hover over an error field with the mouse to display the validation error as a tooltip. Note that it is not possible to apply dialog changes as long as there are sub-conditions with errors.



Logical Operators

A logical operator is rendered as an expandable group in the condition structure. The way to read the structure is to add the operator in between all the listed children. An AND or OR group must have two or more conditions.

The AND group below has two children.

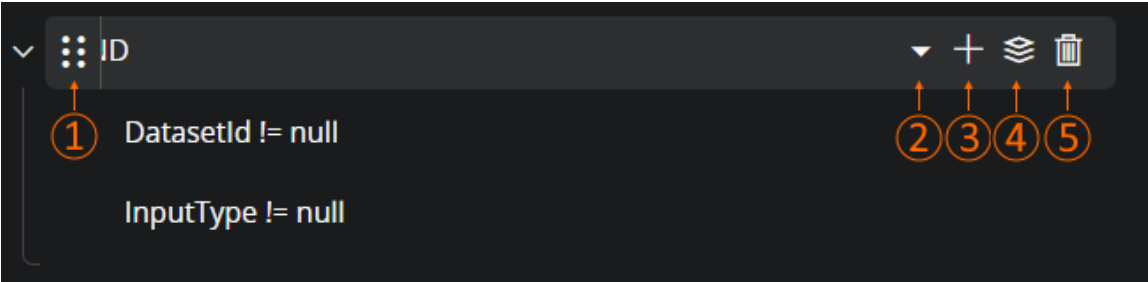
```
// Structure
[AND]
| [DatasetId != null]
| [InputType != null]

// Serialized
DatasetId != null AND InputType != null
```

Nested logical groups follow the same pattern

```
// Structure
[OR]
| [AND]
|   | [DatasetId != null]
|   | [InputType != null]
|   | [released = true]

// Serialized
DatasetId != null AND InputType != null OR released = true
```

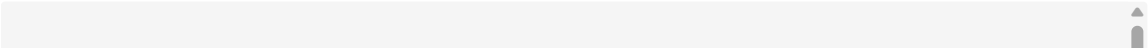


	Description
1	Drag handle, can be used to re-locate a logical group
2	Type picker, to switch between AND OR logical operator
3	Add new child node.
4	Duplicates a logical group including children
5	Deletes a logical group including children

Not Operator

The 'not' operator, also known as the negation operator, is a logical operator used to invert the truth value of a condition. Its purpose is to negate or reverse the value of a boolean expression. In the condition editor, the 'not' operator can be used in a condition or as a group. The difference lies in whether the whole group or just the value of a single operand should be negated. A 'not' operator can only have one child; this child can be an 'and' or an 'or' group, though.

A NOT group will negate the contained structure as



```
[NOT]
  | [open = false]

// Serialized
NOT (open = false)
```

```
[NOT]
  | [AND]
  |   | [DatasetId != null]
  |   | [InputType != null]

// Serialized
NOT (DatasetId != null AND InputType != null)
```

To negate a operand value, write the not in front of the operand within the condition.

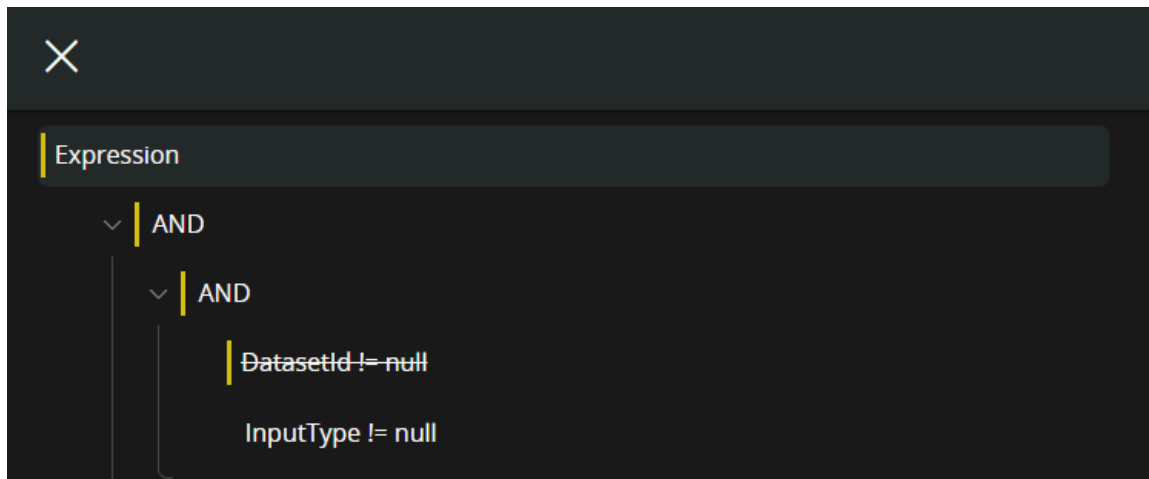
```
[NOT Open = false]

// Serialized
NOT Open = false
```

Expression validation

There are situations where all the sub-conditions validate correctly, but the boolean expression has structural errors. These errors are displayed at the bottom of the editor. Dialog changes cannot be applied until these errors have been corrected.

In the example below, one of the two sub-conditions to an AND logical group is removed, resulting in an invalid expression with a dangling AND operator.



Invalid syntax: The logical operator 'AND' requires at least two operands

Apply

Cancel

Action Examples

Create simple condition

```
[Expression]
  [DatasetId != null]
```

1. Open the condition editor
2. Hover the expression node, click (+) and select 'Condition'
3. Write the condition of choice, e.g., DatasetId != null

Add a logical AND to the simple condition

```
[Expression]
  [AND]
    [DatasetId != null]
    [InputType != null]
```

1. Hover the expression node, click (+) and select 'Logical Operator' AND is the default value for a 'Logical Operator'
2. Drag the 'Condition' by dragging the drag handle, and drop on the 'Logical Operator' node.
3. Hover the 'Logical Operator' node, click (+) and select 'Condition'
4. Write the second condition, e.g., InputType != null

NOTE: You can add as many conditions as you need to the 'Logical Operator'.

Add a logical OR operator

```
[Expression]
  [OR]
    [AND]
      [DatasetId != null]
      [InputType != null]
    [Closed]
```

1. Hover the expression node, click (+) and select 'Logical Operator'
2. On the new 'AND' operator, drop down list and select 'OR'
3. Drag the previous 'AND' operator into the 'OR' operator.
4. Hover the 'OR' operator, click (+) and select 'Condition'
5. Write the condition, e.g., Closed

Change order of execution

In this case it is found that the execution of the condition is much more performant if the check for Closed is done first. Move the Closed condition to the top of the OR group.

```
[Expression]
  [OR]
    [Closed]
    [AND]
      [DatasetId != null]
      [InputType != null]
```

1. Drag the 'Closed' condition by dragging the drag-handle, to the wanted position.